

Operations Research

Polynomial-time solvable integer optimization problems

Prof. Dr. Martin Bichler

Department of Computer Science

School of Computation, Information, and Technology

Technical University of Munich

Course outline

- **Linear Optimization**

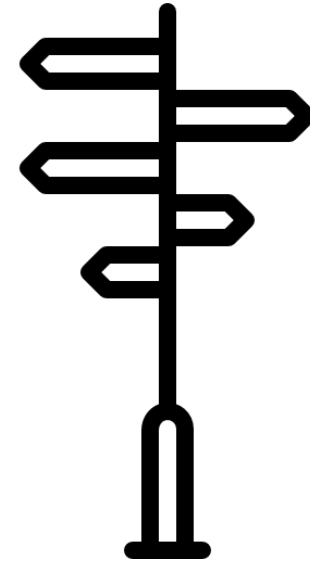
- Modeling by linear programming and graphical solution
- Solving linear programs, convexity
- Simplex algorithm, 2-phase simplex
- Simplex algorithm in matrix notation
- Sensitivity analysis, reoptimization
- Duality theory, zero-sum games

- **Integer Linear Optimization**

- Modeling integer programming problems (IPs)
- Solving IPs: branch & bound, cutting planes
- Advanced solution methods: column generation, approximation algorithms
- **Fast solvable integer problems: total unimodularity, matroids**
- Network flow problems
- Traveling salesperson problem

- **Nonlinear Optimization**

- Optimization of multidimensional functions
- Convex optimization



Today you will learn...

- an **FPTAS** for the knapsack problem
- integer optimization problems that are **easy to solve**
- under which **conditions on the constraint matrix** the continuous relaxation yields integer solutions
- under which **combinatorial structure even simple greedy methods** lead to optimal solutions



Complexity classes relevant for approximation

FPTAS / PTAS

arbitrarily good approximation

APX

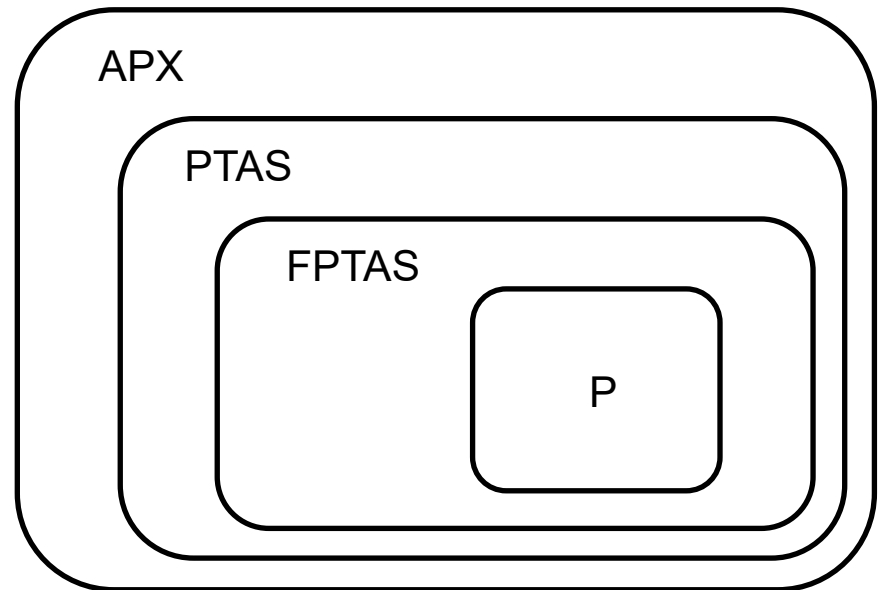
constant-factor approximation exists

APX-complete

constant-factor approximation exists,
but no PTAS unless $P = NP$

Not in APX

no constant-factor approximation
(e.g., maximum clique, set cover, max independent set)



An FPTAS for the knapsack problem

FPTAS: ε -approximation algorithm for each constant $\varepsilon > 0$.

- Produces arbitrarily good approximation guarantee.
 - Algorithm terminates in time polynomial in n and $1/\varepsilon$.
 - Trade-off between optimality and time.
 - Such algorithms do not always exist.
-
- In the following, we show an FPTAS for the knapsack problem.
 - The FPTAS is built on an exact method called **dynamic programming**.

Dynamic programming (DP)

Idea:

Solve a problem by combining subproblems.



Richard E. Bellman
Source: nae.edu

The problem needs a **special structure**:

Each partial solution of an optimal solution must be the optimal solution of its corresponding subproblem!

Bellman's principle of optimality: The combination of the optimal solutions of the subproblems leads to an optimal solution in the end.

- Building on the solution of small subproblems, iteratively larger and larger problems (up to the original problem) are solved.
- There exist slightly different variants of the DP for the knapsack problem. We discuss one.

Example

Capacity (weight) of the knapsack W



i / w	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0					
2	0					
3	0					
4	0					

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

#: (w,v)

// Initialization

for $w = 0$ to W

$B[0,w] = 0$

for $i = 1$ to n

$B[i,0] = 0$

$i := \text{item}, w := \text{weight}$

$B[i,w] := \text{maximal total value that can be achieved with items 1 to } i, \text{ with capacity } w$

Example

i / w	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0				
2	0					
3	0					
4	0					

$$i = 1$$

$$v_i = 3$$

$$w_i = 2$$

$$w = 1$$

$$w - w_i = -1$$

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

if $w_i \leq w$ // item i can be part of the solution

if $v_i + B[i-1, w-w_i] > B[i-1, w]$

$$B[i, w] = v_i + B[i-1, w - w_i]$$

else

$$B[i, w] = B[i-1, w]$$

else **$B[i, w] = B[i-1, w]$** // $w_i > w$

Example

i / w	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3			
2	0					
3	0					
4	0					

$$i = 1$$

$$v_i = 3$$

$$w_i = 2$$

$$w = 2$$

$$w - w_i = 0$$

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

if $w_i \leq w$ // item i can be part of the solution

if $v_i + B[i-1, w-w_i] > B[i-1, w]$

$$B[i, w] = v_i + B[i-1, w - w_i]$$

else

$$B[i, w] = B[i-1, w]$$

else $B[i, w] = B[i-1, w]$ // $w_i > w$

Example

i / w	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	
2	0					
3	0					
4	0					

$$i = 1$$

$$v_i = 3$$

$$w_i = 2$$

$$w = 4$$

$$w - w_i = 2$$

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

if $w_i \leq w$

if $v_i + B[i-1, w-w_i] > B[i-1, w]$

$$B[i, w] = v_i + B[i-1, w - w_i]$$

else

$$B[i, w] = B[i-1, w]$$

else $B[i, w] = B[i-1, w] // w_i > w$

Example

i / w	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0					
3	0					
4	0					

$$i = 1$$

$$v_i = 3$$

$$w_i = 2$$

$$w = 5$$

$$w - w_i = 3$$

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

if $w_i \leq w$

if $v_i + B[i-1, w-w_i] > B[i-1, w]$

$$B[i, w] = v_i + B[i-1, w - w_i]$$

else

$$B[i, w] = B[i-1, w]$$

else $B[i, w] = B[i-1, w] \ // \ w_i > w$

Example

i / w	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	
3	0					
4	0					

$$i = 2$$

$$v_i = 4$$

$$w_i = 3$$

$$w = 4$$

$$w - w_i = 1$$

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

if $w_i \leq w$ // item i can be part of the solution

if $v_i + B[i-1, w-w_i] > B[i-1, w]$ // if $4 + B[1, 1] > B[1, 4]$

$B[i, w] = v_i + B[i-1, w - w_i]$ // $B[2, 4] = 4 + B[1, 1]$

else

$B[i, w] = B[i-1, w]$

else $B[i, w] = B[i-1, w]$ // $w_i > w$

Example

i / w	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0					
4	0					

$$i = 2$$

$$v_i = 4$$

$$w_i = 3$$

$$w = 5$$

$$w - w_i = 2$$

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

if $w_i \leq w$

if $v_i + B[i-1, w-w_i] > B[i-1, w]$

// if $4 + B[1, 2] > B[1, 5]$

$B[i, w] = v_i + B[i-1, w - w_i]$

else

$B[i, w] = B[i-1, w]$

else $B[i, w] = B[i-1, w]$ // $w_i > w$

Example

i / w	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4	5	7
4	0	0	3	4	5	7

$$i = 4$$

$$v_i = 6$$

$$w_i = 5$$

$$w = 5$$

$$w - w_i = 0$$

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

if $w_i \leq w$ // item i can be part of the solution

if $v_i + B[i-1, w-w_i] > B[i-1, w]$

// if $6 + B[3, 0] > B[3, 5]$

$B[i, w] = v_i + B[i-1, w - w_i]$

else

$B[i, w] = B[i-1, w]$

else $B[i, w] = B[i-1, w]$ // $w_i > w$

The maximal value that the knapsack can have is 7!

Pseudocode

```

for w = 0 to W { // Initialize the first row with 0s
    B[0,w] = 0
}
for i = 1 to n { // Initialize the first column with 0s
    B[i,0] = 0
}
for i = 1 to n {
    for w = 0 to W {
        if wi ≤ w { // item i can be part of the solution
            if vi + B[i-1,w-wi] > B[i-1,w]
                B[i,w] = vi + B[i-1,w-wi]
            else
                B[i,w] = B[i-1,w]
        }
        else B[i,w] = B[i-1,w] // wi > w
    }
}

```

For comparison: the brute force approach considers all subsets, i.e. $O(2^n)$

DP for the knapsack problem

The optimal value can be computed recursively:

$$\begin{aligned} B[k, w] &= B[k - 1, w], \text{ if } w_k > w \\ &= \max \{ B[k - 1, w], B[k - 1, w - w_k] + v_k \}, \text{ otherwise} \end{aligned}$$

The best subset with weight w is either:

- the best subset of all previous items with weight w .
- the best subset of all previous objects with weight $w - w_k$ plus item k .

Theorem:

Dynamic programming calculates the optimal value in $O(nW)$.

The algorithm is **pseudo-polynomial** because it is a polynomial in the numerical value of the input (W).

Variants of DP

There is another variant of the DP, in which one builds up the table according to the value and number of items. A cell entry is then the **minimum total weight** needed to achieve a value of exactly v with the items.

- The running time is then $O(nV)$, where **V is an upper bound on the optimal value** of the problem. It holds $V \leq nv_{max}$, i.e., V is at most n times the largest value of an object.
- Thus, we can also write the runtime as $O(n^2 v_{max})$.

Knapsack problem: FPTAS

- DP scales when the values are large integers: $O(n^2 v_{max})$.
- For large instances: scale (and round) the values.
- Run dynamic programming on the rounded instance.
- The rounding can cause errors – hence, the solution is just an approximation!

Item	Value	Weight
1	934.221	1
2	5.956.342	2
3	17.810.013	5
4	21.217.800	6
5	27.343.199	7

Original instance

Item	Value	Weight
1	0	1
2	5	2
3	17	5
4	21	6
5	27	7

Scaled instance

Knapsack problem: FPTAS

Round all values:

- v_{max} = maximum value in the original instance
- ε = precision parameter $0 < \varepsilon < 1$
- θ = scale factor = $\varepsilon v_{max} / n$.

$$\theta = 1.000.000$$

$$\varepsilon = \frac{\theta n}{v_{max}} = \frac{5.000.000}{27.343.199} = 0.1828$$

Round the values $v_i^* = \left\lfloor \frac{v_i}{\theta} \right\rfloor$ and then solve the dynamic program.

Largest value in the scaled version: $\frac{v_{max}}{\theta} = \frac{v_{max}}{\frac{\varepsilon v_{max}}{n}} = \frac{n}{\varepsilon}$

Previous runtime $O(n^2 v_{max})$ from slide 16.

We replace v_{max} in $O(n^2 v_{max})$ with the scaled version $\frac{n}{\varepsilon}$ and obtain $O\left(\frac{n^3}{\varepsilon}\right)$, a runtime that is independent of v_{max} .

If $\varepsilon = 0$, the runtime would be infinite! Thus, a small ε leads to a long runtime.

Rounding the values leads to errors.

Knapsack problem: FPTAS

Theorem:

For each $\varepsilon > 0$, the approximation algorithm returns a feasible solution with value at least $(1 - \varepsilon)V_{opt}$ in $O(n^3/\varepsilon)$ time.

The DP algorithm is an FPTAS for the 0/1 knapsack problem.

→ The runtime is a polynomial in n and $1/\varepsilon$, so it does not depend on v_{max} anymore.

The error of the approximation algorithm is at most $n\theta$, because we can make the error for all n instances.

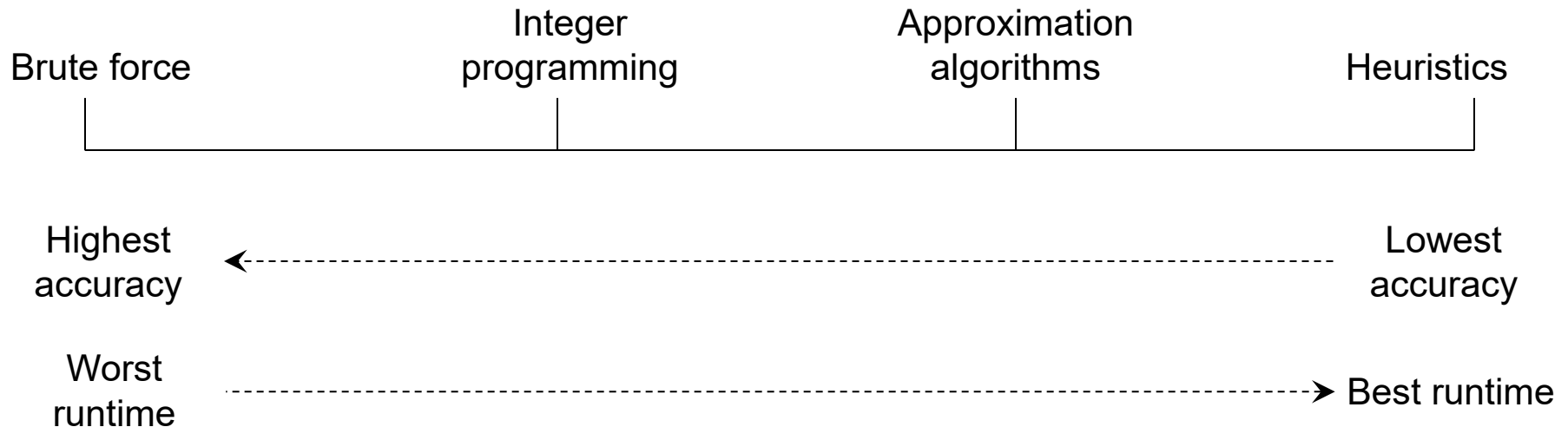
$$V_{approx} \geq V_{opt} - n\theta = V_{opt} - n \cdot \frac{v_{max}}{n} \varepsilon = V_{opt} - \varepsilon v_{max} \geq V_{opt} - \varepsilon V_{opt} = (1 - \varepsilon)V_{opt}$$

In the worst case,
the error is $n \cdot \theta$,
bc. we round down.

$v_{max} \leq V_{opt}$
Largest value of the instance $\leq$ opt. solution

Summary

For some NP-hard problems there exist efficient methods to compute the optimal value at least approximately. The approximation quality varies depending on the problem.



Today you will learn...

- an **FPTAS** for the knapsack problem
- integer optimization problems that are **easy to solve**
- under which **conditions on the constraint matrix** the continuous relaxation yields integer solutions
- under which **combinatorial structure even simple greedy methods** lead to optimal solutions



Total unimodularity

We prove later:

Theorem:

If A is a totally unimodular matrix and b is integral, then $P = \{Ax \leq b, x \in \mathbb{R}_+^n\}$ is an integral polyhedron, and therefore, each vertex is integral.

Definition:

A matrix A is **totally unimodular (TU)** if each square submatrix of A has a determinant with value in $\{-1, 0, 1\}$.

(In particular, all entries are in $\{-1, 0, 1\}$).

Recap: calculating the determinants

2x2 matrices:

$$\begin{vmatrix} a & b \\ c & d \end{vmatrix} = ad - bc$$

3x3 matrices: (Use Laplace expansion or the rule of Sarrus)

$$\begin{vmatrix} a & b & c \\ d & e & f \\ g & h & i \end{vmatrix} = a \begin{vmatrix} e & f \\ h & i \end{vmatrix} - b \begin{vmatrix} d & f \\ g & i \end{vmatrix} + c \begin{vmatrix} d & e \\ g & h \end{vmatrix}$$

Examples

$$\left| \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix} \right| = 2 \quad \left| \begin{pmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \end{pmatrix} \right| = 1 - (-1) = 2 \quad \rightarrow \text{not TU!}$$

$$\left| \begin{pmatrix} 1 & 1 & 0 \\ 1 & 1 & -1 \\ 1 & 0 & 1 \end{pmatrix} \right| = 1 - 2 = -1 \quad \text{contains} \quad \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix} \rightarrow \text{not TU!}$$

$$\begin{pmatrix} 1 & -1 & -1 & 0 \\ -1 & 0 & 0 & 1 \\ 0 & 1 & 0 & -1 \\ 0 & 0 & 1 & 0 \end{pmatrix} \quad \text{TU!}$$

More examples

The identity matrix $I = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$

If A is totally unimodular, then

- $-A$
- A^T
- A^{-1}
- $[A, I]$

are totally unimodular.

$$\begin{pmatrix} 1 & -1 & -1 \\ -1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix} \longrightarrow \begin{pmatrix} 1 & -1 & -1 & 1 & 0 & 0 \\ -1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 \end{pmatrix}$$

TU!

TU!

Example without total unimodularity

$$\begin{pmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \end{pmatrix}$$

Is not totally unimodular,
because determinant = 2.

$$\begin{pmatrix} 0.5 & -0.5 & 0.5 \\ 0.5 & 0.5 & -0.5 \\ -0.5 & 0.5 & 0.5 \end{pmatrix}$$

Inverse

$$\begin{pmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} \longrightarrow \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 0.5 \\ 0.5 \\ 0.5 \end{pmatrix}$$

x is not an integral solution.

Example with totally unimodular basis matrix

$$\begin{pmatrix} 1 & -1 & -1 \\ -1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}$$

B is totally unimodular.

$$\begin{pmatrix} 0 & -1 & 0 \\ 0 & 0 & 1 \\ -1 & -1 & -1 \end{pmatrix}$$

Inverse B^{-1}

$$\begin{pmatrix} 0 & -1 & 0 \\ 0 & 0 & 1 \\ -1 & -1 & -1 \end{pmatrix} \begin{pmatrix} b_1 \\ b_2 \\ b_3 \end{pmatrix} = \begin{pmatrix} -b_2 \\ b_3 \\ -(b_1 + b_2 + b_3) \end{pmatrix}$$

$B^{-1}b$ is integral
if b is integral.

Cramer's rule

Cramer's rule can be used to solve linear systems of equations.

For a system $Ax = b$, where A is a regular $m \times m$ matrix, the solution for the i -th variable is:

$$x_i = \frac{\det(A_i)}{\det(A)}$$

- where the matrix A_i is identical to the matrix A , but the i -th column is replaced with the vector b .

Example

$$A = \begin{pmatrix} 3 & 1 \\ 1 & 1 \end{pmatrix}, b = \begin{pmatrix} 7 \\ 3 \end{pmatrix}$$

$$x_1 = \frac{|A_1|}{|A|} = \frac{\begin{vmatrix} 7 & 1 \\ 3 & 1 \end{vmatrix}}{\begin{vmatrix} 3 & 1 \\ 1 & 1 \end{vmatrix}} = \frac{4}{2} = 2$$

$$x_2 = \frac{|A_2|}{|A|} = \frac{\begin{vmatrix} 3 & 7 \\ 1 & 3 \end{vmatrix}}{\begin{vmatrix} 3 & 1 \\ 1 & 1 \end{vmatrix}} = \frac{2}{2} = 1$$

Totally unimodular matrices

Theorem:

If A is a total unimodular matrix and b is integral, then $P = \{Ax \leq b, x \in \mathbb{R}_+^n\}$ is an integral polyhedron and, therefore, each vertex is integral.

Proof:

- Each vertex x^* is the unique solution to $x_N^* = 0$ and $Bx_B^* = b$.
- If A is totally unimodular and B is a regular matrix (i.e., $\det(B) \neq 0$) then $\det(B) = \pm 1$.
- By Cramer's rule we know that $x_{B_i}^* = \det(B_i) / \det(B)$ for all i .
- If B and b are integral, then the determinant $\det(B_i)$ is integral.
- $x_B^* = \det(B_i) / \pm 1$ is, thus, integral.
- Appending identity rows preserves total unimodularity.

Example: Rostering

Time period	Time frame				Minimum number of required workers
	Shift				
	1	2	3	4	
6:00 to 9:00	x				48
9:00 to 12:00	x	x			79
12:00 to 15:00	x	x			65
15:00 to 18:00		x	x		87
18:00 to 21:00			x		64
21:00 to 00:00			x	x	73
00:00 to 6:00				x	82

Let $A \in \{0,1\}^{m \times n}$ be a matrix. We say that A satisfies the *consecutive ones property*, if there exists a permutation matrix $P \in \{0,1\}^{m \times m}$, such that the 1s in each column of $P \cdot A$ are directly consecutive. In this case, A is totally unimodular.

Sufficient conditions for total unimodularity

A matrix A is totally unimodular, if

- 1) a_{ij} in $\{+1, -1, 0\}$ for all i, j .
- 2) each column has at most 2 coefficients that are not 0.
- 3) there exists a partition (M_1, M_2) of the set M of rows, such that
 - if 2 non-zero entries in a column of A have the same sign, then the row of one entry is in M_1 , the other in M_2 .
 - if 2 non-zero entries in a column of A have different signs, both rows are either in M_1 or in M_2 .

Example: Transportation problem

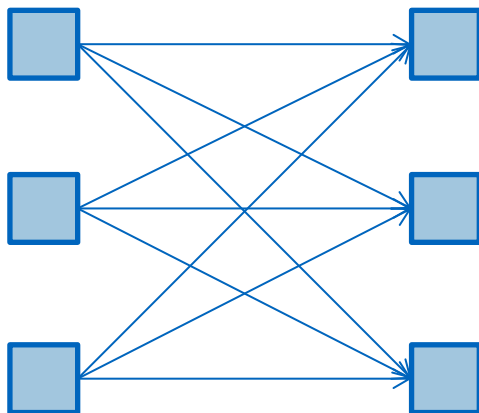
$$\begin{array}{cccccccc}
 x_{11} & x_{12} & x_{13} & x_{21} & & \dots & & x_{33} \\
 \left(\begin{array}{cccccccc}
 -1 & -1 & -1 & 0 & 0 & 0 & 0 & 0 \\
 0 & 0 & 0 & -1 & -1 & -1 & 0 & 0 \\
 0 & 0 & 0 & 0 & 0 & 0 & -1 & -1 \\
 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 \\
 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 \\
 0 & 0 & 1 & 0 & 0 & 1 & 0 & 1
 \end{array} \right)
 \end{array}$$

$$\begin{array}{ll}
 \min & \sum_i^n \sum_j^m c_{ij} x_{ij} \\
 \text{s.t.} &
 \end{array}$$

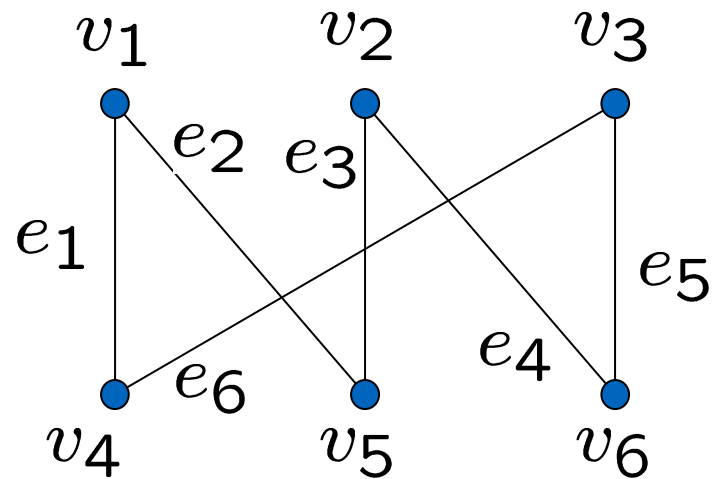
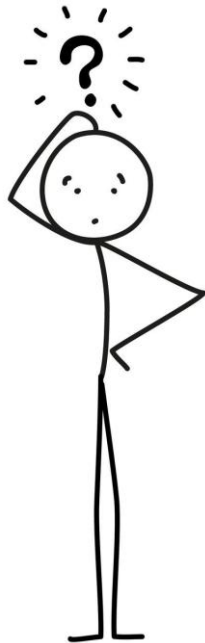
$$\sum_j^m -x_{ij} \geq -u_i \quad \text{for all } i = 1, \dots, n$$

$$\sum_i^n x_{ij} \geq l_j \quad \text{for all } j = 1, \dots, m$$

$$x_{ij} \geq 0$$



Can you construct the incidence matrix for the following bipartite graph?



Graph theory – a short recap

Many real world problems can be represented as graphs.
Such problems can often be solved with (Integer) Linear Programming.

Definition **(undirected) Graph**:

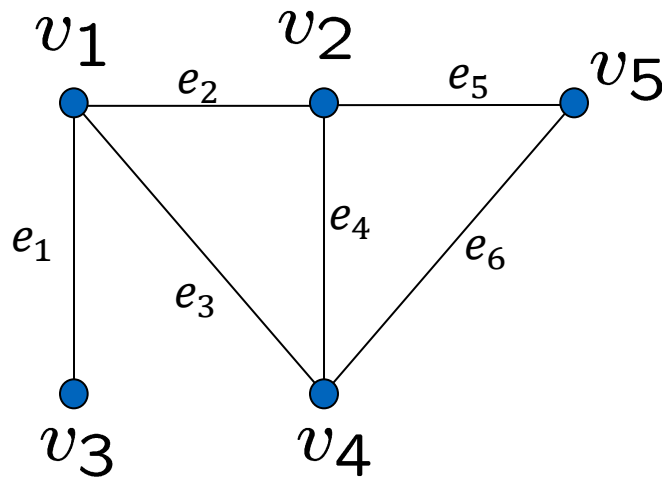
- A graph G is a pair (V, E) .
- V is the finite set of vertices (nodes).
- $E \subseteq \{\{a, b\} | a, b \in V, a \neq b\}$ is the set of edges.

Definition **Directed graph (Digraph)**

- A directed graph $G = (V, E)$ consists of a finite set of vertices V and a set E of ordered pairs (start and end vertices of an edge).
- $E \subseteq V \times V = \{(a, b) | a, b \in V\}$

Incidence matrix

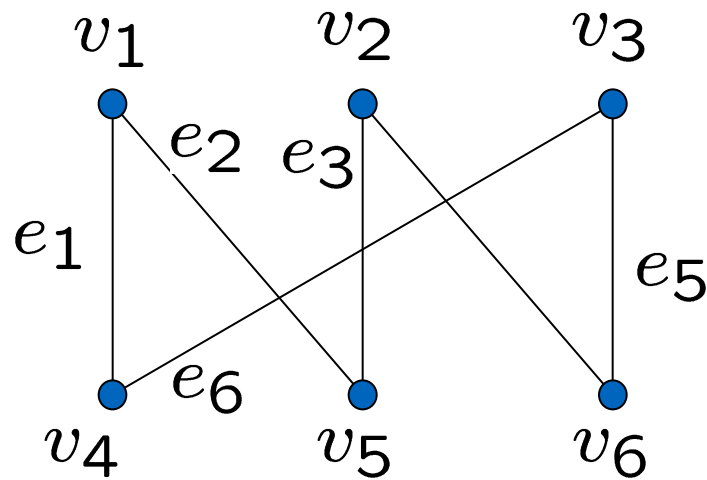
- Vertices a and b are **adjacent** or **neighboring**
 - in undirected case: if $\{a, b\} \in E$
 - in directed case: if $(a, b) \in E$ or $(b, a) \in E$.
- The vertices a and b are then **incident** to the edge e .
- Incidence matrix (in undirected case):
 - Each row i represents a vertex v_i and each column j represents an edge e_j .
 - $a_{ij} = 1$ iff v_i is incident to e_j .



Vertices	Edges					
	1	1	1	0	0	0
	0	1	0	1	1	0
	1	0	0	0	0	0
	0	0	1	1	0	1
	0	0	0	0	1	1

TU: Bipartite graphs

A graph $G = (V, E)$ is bipartite, if its nodes can be partitioned into two disjoint sets $M_1 \uplus M_2 = V$, such that no nodes within the sets M_1, M_2 are adjacent to each other.



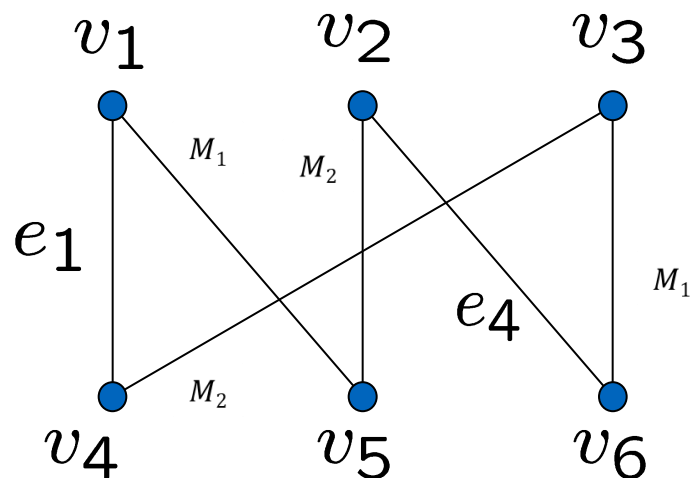
Vertices	Edges					
	1	1	0	0	0	0
	0	0	1	1	0	0
	0	0	0	0	1	1
	1	0	0	0	0	1
	0	1	1	0	0	0
	0	0	0	1	1	0

TU: Undirected bipartite graphs

Let A be the incidence matrix of a bipartite graph (without self loops at the vertices).
Each column of A has exactly two entries equal to 1.

We can reformulate A as: $A = \begin{pmatrix} M_1 \\ M_2 \end{pmatrix}$

Since the graph is bipartite, each column has a 1 in M_1 and a 1 in M_2 .

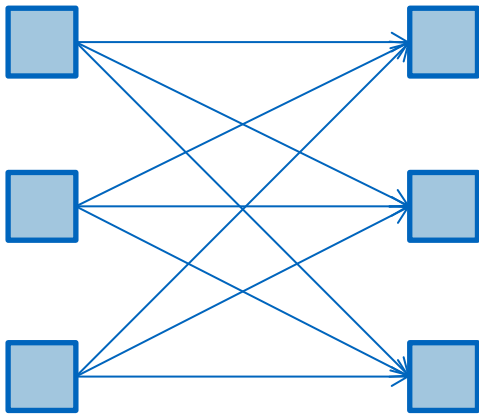


		Edges						
Vertices	$\left(\begin{array}{cccccc} 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 \end{array} \right)$	}			M_1			
		}			M_2			

TU: Directed graphs

Incidence matrix (in directed case, without self loops at the vertices):

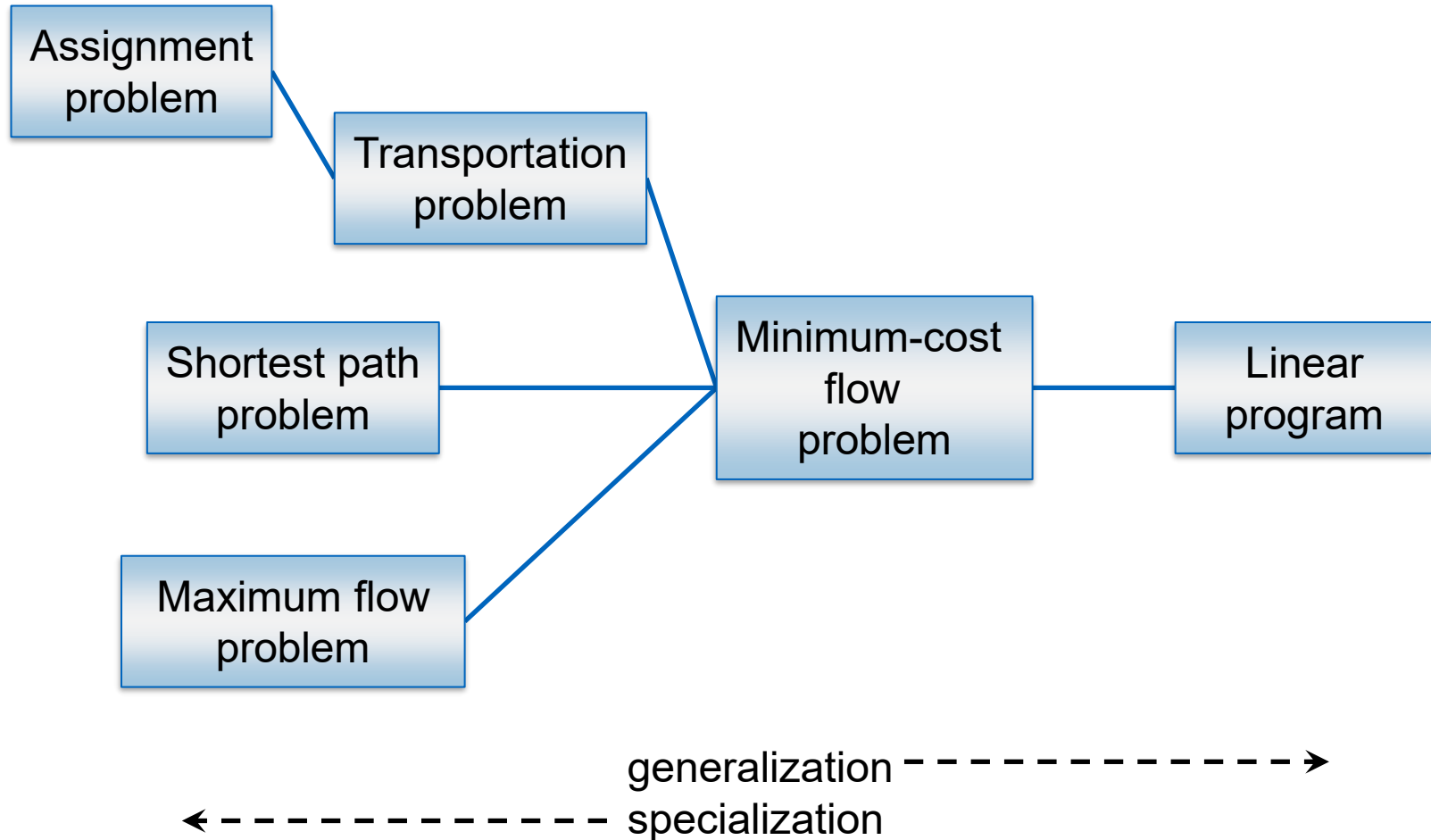
- Each row i represents a node v_i and each column j represents an edge e_j .
- If $e_j = (k, i)$ (i.e. k is incident to i), then $a_{kj} = -1$ (outgoing edge) and $a_{ij} = +1$ (ingoing edge).



$$\begin{pmatrix} -1 & -1 & -1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & -1 & -1 & -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & -1 & -1 & -1 \\ 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 \end{pmatrix}$$

Always different signs, M_2 is empty!

Some network problems that can be solved efficiently



Graph problems as LPs

Many graph problems can be solved using LP.

Often the constraint matrix is the incidence matrix (so it is TU!) or differs only slightly and remains TU.

Examples:

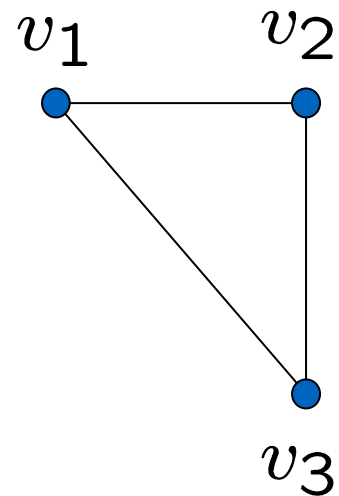
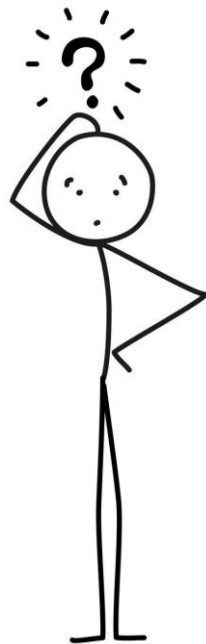
- Transportation problem
- Matching in bipartite graphs
- Computing shortest paths
- Network flow problems

However, for many of these graph and network flow problems, there are specialized, faster algorithms (faster than LP methods).



Next unit

Is the incidence matrix of the following graph TU?



Matroids

Matroids are combinatorial structures that generalize the property of linear independence in linear algebra.

If an (integer) optimization problem has this combinatorial structure, the solution is often easier to find.

- with LP
- even stronger: even with a **greedy algorithm** we find an optimal solution.

Greedy algorithm:

- Simple procedure that chooses the best option determined by the algorithm in each iteration.
- Example: Heuristics in the knapsack problem from the last unit.

Matroid definition

Let E be a finite set and I a non-empty set of subsets of E ,
i.e. $I \subseteq \wp(E)$ (power set of E).

(E, I) is a **Matroid** if:

- 1) $\emptyset \in I$, in particular, I is not empty.
- 2) (Hereditary property) If $B \in I$ and $A \subseteq B$, then $A \in I$.
- 3) (Exchange property) If $A, B \in I$ and $|A| < |B|$, then there exists an $x \in B \setminus A$ such that $A \cup \{x\} \in I$.

E is called the **ground set**.

The elements in I are called **independent sets**.

A maximal independent set is called a **basis**.

Example

$$E = \{a, b, c, d\}.$$

Construct a matroid (E, I) such that $\{a, b\}$ and $\{c, d\}$ are included in I and I is minimal.

$$I = \{\emptyset, \{a\}, \{b\}, \{c\}, \{d\}, \{a, b\}, \{c, d\}, \text{ via properties 1 and 2}$$

$$\{a, c\}, \{b, d\} \text{ via property 3 } \}$$

$$|\{a\}| < |\{c, d\}| \Rightarrow \{a, c\}, |\{b\}| < |\{c, d\}| \Rightarrow \{b, d\}, |\{c\}| < |\{a, b\}| \Rightarrow \{a, c\}, |\{d\}| < |\{a, b\}| \Rightarrow \{b, d\}$$

or

$$I = \{\emptyset, \{a\}, \{b\}, \{c\}, \{d\}, \{a, b\}, \{c, d\}, \text{ via properties 1 and 2}$$

$$\{a, d\}, \{b, c\} \text{ via property 3 } \}$$

Property 3 (Exchange property):

If $A, B \in I$ and $|A| < |B|$, then there exists an $x \in B \setminus A$, such that $A \cup \{x\} \in I$.

Matroids in vector spaces

Let M be a matrix and E a set of columns of M .

$$I = \{A \mid A \subseteq E, A \text{ is linearly independent} \}$$

Claim: (E, I) is a matroid.

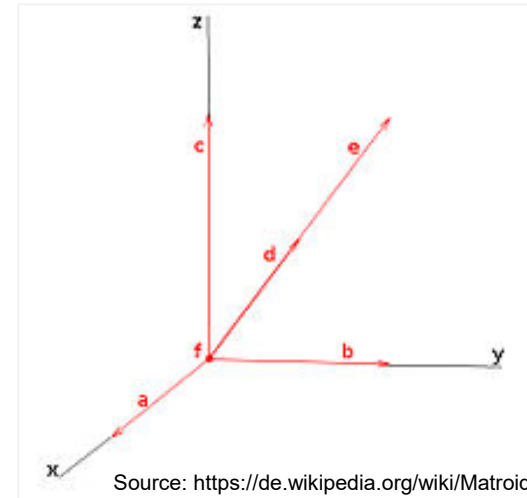
- 1) The empty set of columns is independent.
- 2) If B is a set of linearly independent columns of M , then every subset A of B is linearly independent (hereditary).
- 3) If A, B are sets of linearly independent columns of M , and $|A| < |B|$, then $\dim(\text{span}(A)) < \dim(\text{span}(B))$.
[$\text{span}(A)$ is the space of all vectors expressed as a linear combination of the columns]

Choose a column $x \in B$ that is not contained in $\text{span}(A)$.

Then $A \cup \{x\}$ is a linearly independent subset of columns of M .

Vector matroid

$$A := \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & \frac{1}{2} & 1 & 0 \\ 0 & 0 & 1 & \frac{1}{2} & 1 & 0 \end{pmatrix}$$



$$a := \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, b := \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}, c := \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}, d := \begin{pmatrix} 0 \\ \frac{1}{2} \\ \frac{1}{2} \end{pmatrix}, e := \begin{pmatrix} 0 \\ 1 \\ 1 \end{pmatrix}, f := \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$$

$$E = \{a, b, c, d, e, f\}$$

$$I = \{\emptyset, \{a\}, \{b\}, \{c\}, \{d\}, \{e\}, \{a, b\}, \{a, c\}, \{a, d\}, \{a, e\}, \{b, c\}, \{b, d\}, \{b, e\}, \\ \{c, d\}, \{c, e\}, \{a, b, c\}, \{a, b, d\}, \{a, b, e\}, \{a, c, d\}, \{a, c, e\}\} \subseteq \wp(E)$$

$B = \{\{a, b, c\}, \{a, b, d\}, \{a, b, e\}, \{a, c, d\}, \{a, c, e\}\}$ is the **basis of the vector matroid**, i.e., the maximal linearly independent subset of the given columns.

Note that the zero-vector $\{f\}$ is dependent by itself and can't be an independent set.

Graphic matroid

Let $G = (V, E)$ be an undirected multigraph (i.e. multiple edges are possible) with node set V and edge set E .

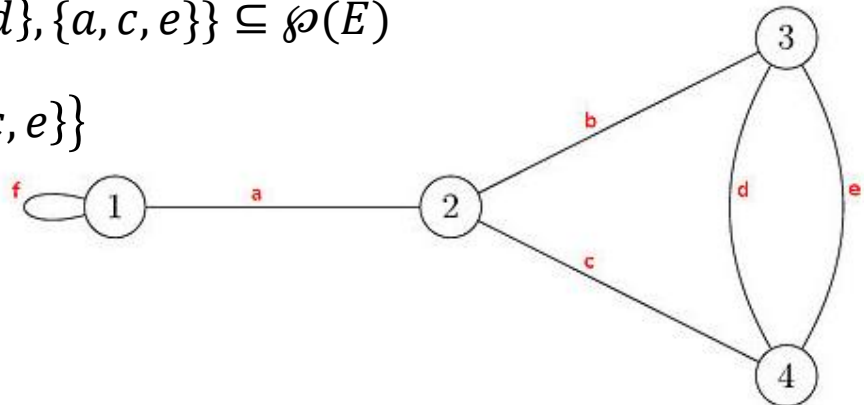
The matroid $M = (E, I)$ contains the cycle-free subgraphs of G , i.e. forests, as independent sets.

The bases B of a graphic matroid **correspond to the spanning trees** of a connected graph.

$$E = \{a, b, c, d, e, f\}$$

$$I = \{\emptyset, \{a\}, \{b\}, \{c\}, \{d\}, \{e\}, \{a, b\}, \{a, c\}, \{a, d\}, \{a, e\}, \{b, c\}, \{b, d\}, \{b, e\}, \{c, d\}, \{c, e\}, \{a, b, c\}, \{a, b, d\}, \{a, b, e\}, \{a, c, d\}, \{a, c, e\}\} \subseteq \wp(E)$$

$$B = \{\{a, b, c\}, \{a, b, d\}, \{a, b, e\}, \{a, c, d\}, \{a, c, e\}\}$$

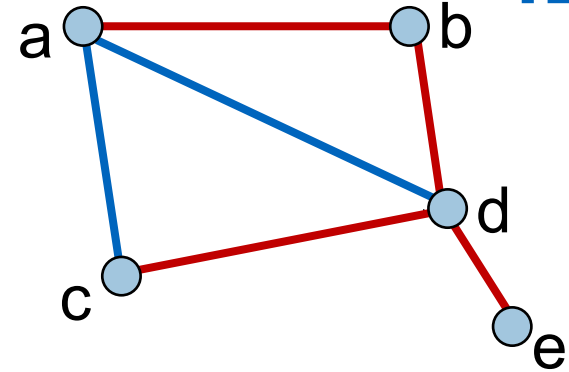


Graphic matroids

Let $G = (V, E)$ be an undirected graph.

I is the set of all edge subsets that do not contain a cycle.

The set of all forests I is an independent system of E .



Claim: (E, I) is a matroid.

1) The empty set is a subset (each node is a tree).

2) Subsets of forests are also forests.

3) Let A and B in I with $|B| < |A|$. Then (V, B) has less edges than (V, A) .

The number of trees in A is larger than the number of trees in B . There must exist an edge in A such that $B \cup \{x\} \in I$.

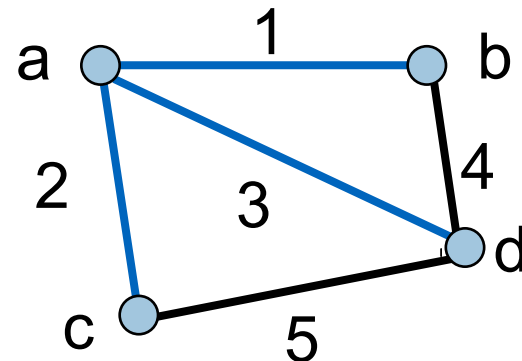
Minimum spanning trees

Let $G = (V, E)$ be a graph. A graph $T = (V', E')$ is a subgraph of G if $V' \subseteq V$ and $E' \subseteq E$.

A subgraph T is a spanning tree of G if $V' = V$ and T is a tree (i.e. connected and cycle-free).

In a spanning tree all nodes are connected to each other!

If the edges are weighted,
then the spanning tree with
the minimum weight is a
minimum spanning tree.



Weight function

A matroid (E, I) is weighted if it has an associated weight function $w: E \rightarrow \mathbb{R}_{\geq 0}$.

If A is an element of I , then $w(A) := \sum_{a \in A} w(a)$.

Weight functions of this form are also called linear weight functions.

Minimum spanning trees as IP

Variables:

- $x_e \in \{0,1\}$ for all $e \in E$: edge e with weight w_e is part of the spanning tree.

Constraints:

- Each subgraph $T = (V', E')$ of the spanning tree $G = (V, X)$ is a forest.
- Sufficient and necessary condition: at most $|V'| - 1$ edges between nodes (rank function of the graphic matroid: the max no. of edges w/o forming a cycle).
- At the end, the tree has $|V| - 1$ edges.

$$\begin{aligned}
 & \min \sum_{e \in E} w_e x_e \\
 & \sum_{\substack{e=(i,j) \in E \\ i,j \in V'}} x_e \leq |V'| - 1 && \text{for all } V' \subset V \\
 & \sum_{e \in E} x_e = |V| - 1 \\
 & x_e \in \{0,1\} && \text{for all } e \in E
 \end{aligned}$$

Greedy algorithms for matroids

Greedy($M = (E, I), w$)

$A := \emptyset$;

Sort E in monotone increasing order w.r.t. weight w .

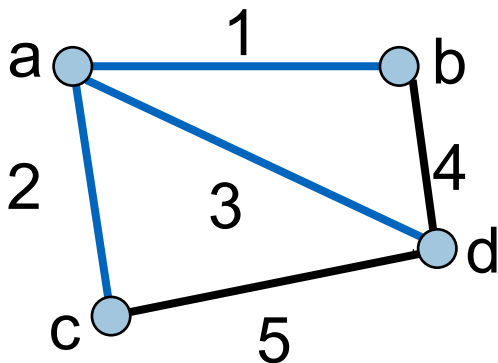
for each $x \in E$ in monotone increasing order **do**:

if $A \cup \{x\} \in I$, **then** $A := A \cup \{x\}$;

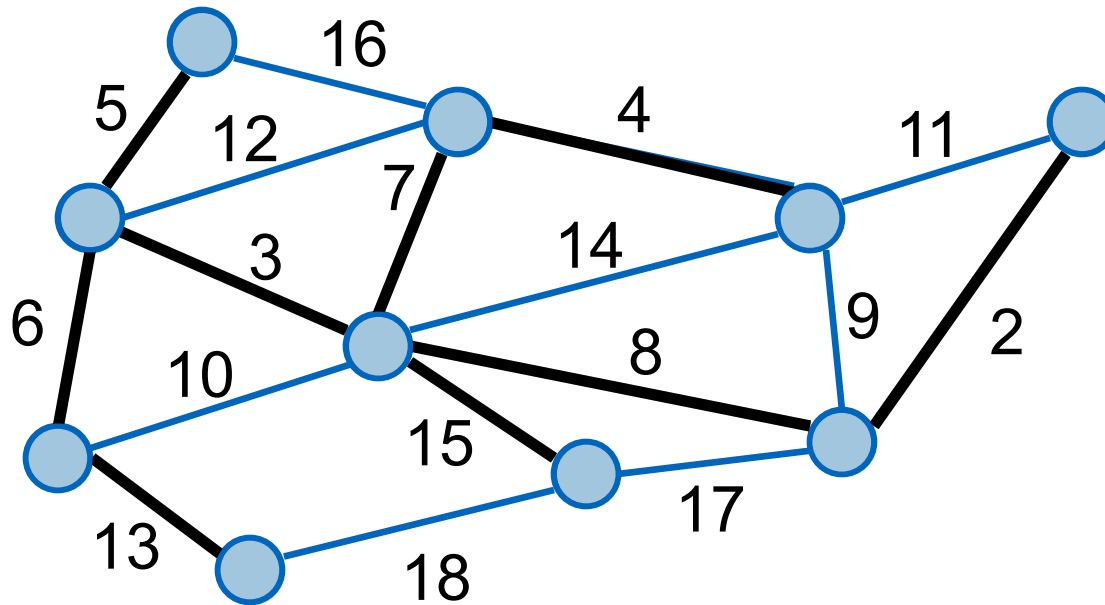
return A ;

Theorem:

Let M is a weighted matroid, then **Greedy**(M, w) in increasing weight order computes a minimum-weight basis.



Kruskal's MST Algorithm



Consider the edges in increasing order w.r.t. weight.
Add an edge if it does not create a cycle.

Runtime of Greedy

Let $n = |E| = \#$ elements in the set E .

Sorting E : $O(n \cdot \log n)$

The for-loop iterates n times. In the loop it must be checked if $A \cup \{x\} \in I$.

If each check takes $f(n)$ time, then the loop takes $O(n \cdot f(n))$ time.

Thus, Greedy has $O(n \cdot \log n + n \cdot f(n))$ time.

Rank function of matroids

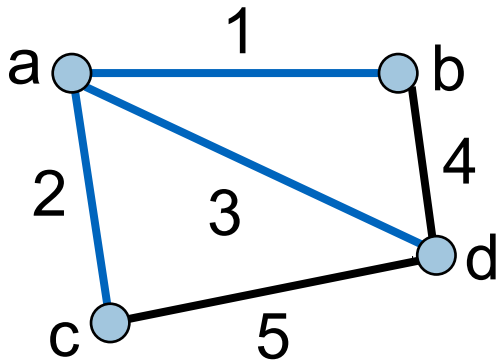
As in the case of matrices, we may define the *rank* of a matroid.

Let (E, I) be a matroid. Then the rank function $r_M: \wp(E) \rightarrow \mathbb{N}_0$

for each $B \subset E$ is defined as $r(B) = \max\{|A| : A \subseteq B, A \in I\}$

Thus, the rank is the **cardinality of the largest independent subset of B** .

Example for the minimum spanning tree:



Rank = 3

The largest independent sets in the graphic matroid are spanning trees with $(|V|-1)$ edges!